



A real-time autonomic architecture for detection and defeat of open-access chess bots

Andrew Bengtson^{1,2} · Ali Tekeoglu^{1,3} · Christopher Rouff^{1,2}

Received: 8 January 2025 / Accepted: 21 September 2025 / Published online: 11 October 2025
© Institut Mines-Télécom and Springer Nature Switzerland AG 2025

Abstract

With the rise of online chess, the increasing prevalence of automated chess bots, and recent high-profile cheating controversies, there is growing interest in developing effective methods for detecting bots in chess. Current approaches in this domain largely depend on analyzing player history, detecting anomalies, and conducting engine analysis to identify bot-like behavior after a game has concluded. However, these post-hoc techniques struggle to adapt to real-time detection scenarios, such as those required in dynamic cybersecurity contexts. This paper introduces a novel challenge: detecting bots during an ongoing game, enabling adaptive strategies based on the real-time identification of an opponent's behavior. It further proposes an autonomic system leveraging self-adaptive properties to address this challenge as well as discussing the application of this bot detection to other domains.

Keywords Chess · Bot detection · Game theory · Reinforcement learning · MAPE-K · Shelf-CHOP · Autonomic computing

1 Introduction

The Turing test is one of the rare concepts from the theory of computation that has permeated popular culture. A notable example is its depiction in the film *Blade Runner*, where audiences are captivated by the idea of sitting face-to-face with an unknown entity and asking questions to determine whether it is human or machine. While many associate this concept solely with science fiction, it is increasingly relevant in everyday life. People routinely face situations where they must discern whether an anonymous actor is human or automated. Examples include determining whether an email

is an automated phishing attempt, evaluating whether online opinions are posted by bots, or assessing whether customer support responses are generated by a chatbot or a human representative. The task of distinguishing human actions from automated ones has become a routine aspect of modern interactions. Moreover, many automated systems have known vulnerabilities that users can exploit, intentionally or accidentally, to achieve more favorable outcomes, as demonstrated in cases where customers manipulate large language models (LLMs) deployed for customer service [1].

In this paper, we build on previous work to tackle the challenge of real-time bot detection and identification by proposing a novel chess scenario [2, 3]. In this setup, a chess bot must compete against an opponent that could randomly be either an open-source chess engine, an unknown chess bot, or a sequence of recorded human chess moves. The bot must analyze its opponent's behavior to gather information without jeopardizing its performance in the game. If the opponent is identified as a specific open-source chess engine, this knowledge can be leveraged to pair it against an adversary it is known to struggle against. The proposed autonomic chess engine will utilize the self-CHOP properties of self-optimization and self-configuration to handle the complexity of this task. It will dynamically assess whether the opponent is a recognized engine and adapt its strategy to maximize its chances of victory. The idea of this challenge

Ali Tekeoglu and Christopher Rouff contributed equally to this work.

✉ Andrew Bengtson
abengts4@jh.edu

Ali Tekeoglu
ali.tekeoglu@jhu.edu

Christopher Rouff
christopher.rouff@jhuapl.edu

¹ Whiting School of Engineering, Johns Hopkins University, Laurel, MD, USA

² Applied Physics Laboratory, Johns Hopkins University, Laurel, MD, USA

³ Leidos Innovations Center (LInC), Columbia, MD, USA

was originally presented at the IEEE CSNet 2024 Conference [2]. This paper expands on our previous work with a sample solution to this challenge, providing a modular autonomic architecture for chess bot detection and defeat, lowering the barrier to entry for those interested in this challenge. This paper also documents findings from testing our sample solution in using the highest-ranked open-source chess engines available.

The rest of the paper is organized as follows: Section 2 will provide a brief background and history of chess and bot detection. Detailed investigation of related work in the literature will be presented in Section 3. Section 4 provides a brief summary of the challenge tackled with the proposed work, while Section 5 discusses the proposed work at a higher level. Section 6 goes into detailed design and implementation of the system. The results of the experiments are discussed in Section 7. Section 8 goes over potential future extensions, and lastly Section 9 concludes the paper.

2 Background

The history of chess and bot detection dates back to 1770 with the creation of a chess-playing automaton known as the “Mechanical Turk.” Claimed by its creator to be entirely mechanical, the automaton famously defeated notable historical figures, including Napoleon and Benjamin Franklin, in chess matches [4, 5]. To a modern audience, it is evident that a human operator was concealed inside the device, as illustrated in Fig. 1. Interestingly, even in the eighteenth

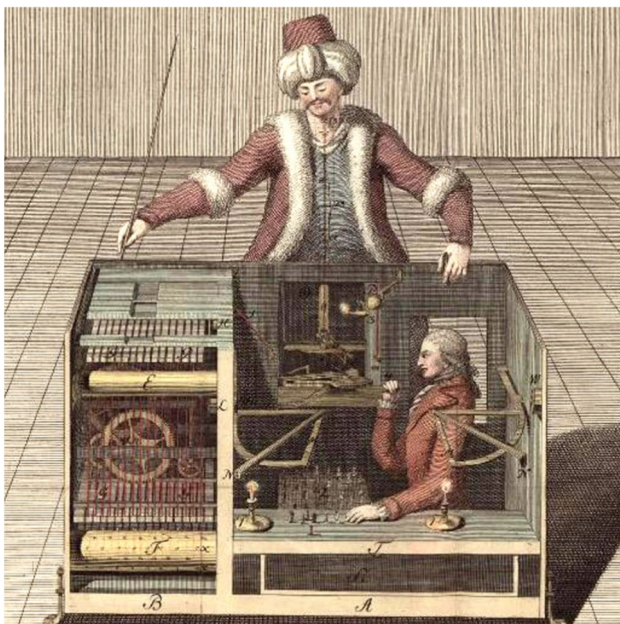


Fig. 1 The illusion of the Mechanical Turk illustrated [6]

century, onlookers speculated about how a person might be hidden within the machine [4]. While this early intrigue could be considered one of the first instances of bot detection in chess, the centuries that followed would witness remarkable advancements in chess automation and the challenges it presents.

Mathematicians have long been intrigued by the pursuit of an optimal chess strategy. Although no definitive mathematical proof for optimal play has yet been established, the advent of computing has led to the development of highly sophisticated *chess engines*—automated systems capable of competing at an exceptional level. The Elo rating system, a widely adopted method for ranking chess players based on tournament performance, has been utilized by numerous national chess federations since its introduction in the 1970s [7].

The Elo rating system, developed by Hungarian-American physicist Arpad Elo, is a widely used method for assessing the relative skill levels of players in competitive games, particularly chess [8]. It assigns each player a numerical rating that adjusts dynamically based on their performance against opponents, with updates reflecting the difference between expected and actual outcomes. Higher-rated players are expected to win more often against lower-rated opponents, and the system calculates the expected outcomes using a logistic curve based on the difference in rating [9]. After each game, a player’s rating is updated using the following formula:

$$R_{new} = R_{old} + K \cdot (Actual - Expected) \quad (1)$$

where the K-factor determines the magnitude of rating changes. Widely adopted by organizations like FIDE since the 1970s, the Elo system is central to chess tournaments for ranking players, determining title eligibility, and creating balanced match pairings. Although highly effective, the system assumes consistent player performance and can lose accuracy for inactive players, yet its adaptability has made it a standard for evaluating skill in chess and beyond [10]. With the increasing code reasoning capabilities of existing large language models (LLMs) and breakthroughs in reasoning models, recently Elo-based ranking systems to test LLMs sophisticated competition-level coding abilities are developed [11].

The average Elo rating of players on [chess.com](https://www.chess.com) is around 1000. Chess Grandmasters generally have Elo ratings above 2500 with the highest Elo rating ever achieved for a human player being Magnus Carlsen with an Elo of 2882 [12]. There are 169 unique chess engines (ignoring different versions of the same engine) that have achieved an Elo rating over 2882, with the 4 highest Elo rated engines all over 3600 [13]. Two of those top 4 engines are entirely open source, with the current

highest Elo rated chess engine, *Stockfish*, being both open source and highly accessible [13].

Another huge way the game of chess has changed over the last few decades has been the popularization of online chess, particularly on the website [Chess.com](https://www.chess.com). With the widespread availability of these open-source chess engines, and play moving increasingly online, there is a huge opportunity for players to cheat by using moves from chess engines or even to allow these engines to play games for them.

3 Related work

There have been a number of approaches to determine if a chess player is cheating by using a chess engine for recommendations. In [14], the authors discuss the impact that artificial intelligence (AI) has had on the Chess world. Early results were obtained by brute force, where chess software was able to search millions of moves per second. Later attempts used reinforcement learning that mimicked human play by learning from a large number of chess games. These advances have been used by online players to help them play above their ability, in effect cheating. To counter this, cheat detection algorithms have been developed that detect unlikely chess moves, in effect, using AI to catch cheaters using AI.

In [15], the author discusses how Chess platforms use statistical models to determine whether a player's move matches a move that a chess engine would make. A high correlation between a player's moves and an engine's moves can indicate potential cheating. Some AI machine learning models have used supervised training to predict human moves in games. These predictions can then be used to determine if a move is highly unlikely for a human to make, but likely for an AI system to make. These models can estimate the probability that a human would make any given move from any given position, thus be used as an aid to detect cheating. Though a move may be highly unlikely for a human, that does not mean that a human would not make that move, thus making any cheating allegations to be possibly unfounded.

In [3], the authors discuss how current cheat detection methods are not transparent, which leads to mistrust in the online chess community. The authors analyzed millions of games using data on time control, player ratings, and chess engine analysis. They also analyzed games from computer chess championships and human-like AI models. Metrics included accuracy and evaluation loss, player response time, move complexity metrics, and clock management. They found that the quality of moves and response times were predictable for humans based on position complexity, remaining time, and player strength. They found that humans have distinct behavioral patterns while playing chess that chess engines cannot fully duplicate. They found that metrics like evaluation loss and response time distributions can be used

to distinguish cheaters from legitimate players, but these metrics are not foolproof. They found that a naïve binary classifier using these metrics achieved some success, but also showed that more sophisticated models are needed. The authors noted that any cheat detection methods used should balance accuracy while minimizing wrongful accusations of cheating. The authors argue that transparency in cheat detection algorithms can improve trust in the online chess community in these algorithms.

In [21], the authors describe chessGPT, a GPT-based model that bridges policy learning and language modeling by integrating data from both sources in chess games. ChessGPT combines historical policy data and analytical insights in natural language. The historical policy data provides interaction replays from the environment, and the natural language provides thought processes and strategic considerations. They have created a dataset that encompasses chess game data and language annotations. The authors have open-sourced their code, models, and dataset to assist others in researching this area.

There is some existing literature on attempting to identify chess engines from recordings of chess games, particularly led by Professor Kenneth Regan who carried out early work into anomaly detection in ranked chess and proposed matching adjustments to the ranking system [16–18]. While this work opened up the field, Regan's cheat detection systems largely rely on detecting anomalies in player performance relative to their previous play. This is clearly a valuable technique for online tournament operators looking to detect bots within a few games; however, for the goal of detecting the bot mid-game, approaches that rely on previous player history have to be largely discounted. For the sake of the proposed work's challenge in this paper, it will be assumed that all games are being played by pairs of first-time players who do not have any player history to utilize.

Further, work has been done attempting to use large chess gameplay datasets to train machine learning (ML) models in order to detect bot play in an online chess games [19]. However, these approaches can be proven to detect false positives, as demonstrated by Barnes and Hernandez-Castro who received many engine "detections" when applying previously published engine detection techniques on ranked games played in person [20]. This is obviously highly concerning when allegations of foul play are being leveled based on these analytical techniques; however, for this project's challenge, the issue of false positive bot detections is not particularly concerning as it is built into the reward scheme of the game. If the autonomic chess bot detects the Stockfish chess engine when it is in fact playing against the Obsidian engine, the autonomic bot will simply play non-optimally against the opponent it is truly facing, which will likely result in a loss of the game. This is in and of itself a bad outcome for the autonomic chess bot's reward function, mitigating the

Table 1 Comparison to related research

Reference#	Online detection	Limitation	Technique
Regan et al. [16]	✗	Needs previous play history	Statistical methods
Di Fatta et al. [17]	✗	Needs previous play history	Bayesian inference
Haworth et al. [18]	✗	Needs previous play history	Bayesian inference
Patria et al. [19]	✓	Needs large dataset for ML training	Neural networks
Barnes et al. [20]	✗	Needs previous play history	Statistical methods
Laarhoven et al. [3]	✓	Needs large dataset for ML training	Statistical methods
Proposed work	✓	✗	Autonomic

likelihood of excessive false positives. Overall comparison of the related work with our proposed method is summarized in Table 1.

Another key piece of background for this project comes from the field of autonomic computing. In October of 2001, the Senior Vice President of IBM Research, Paul Horn wrote “Autonomic Computing: IBM’s Perspective on the state of Information Technology” [22] as a manifesto for the future of computing. Horn’s vision for autonomic systems suggests that the ever-growing complexity of modern computing systems can be managed best by allowing these systems to manage themselves at a low level under high-level human guidance. This vision was later refined by Kephart and Chess [23] who envisioned an autonomic system as a system implementing the four self-CHOP properties, namely self-configuration, self-healing, self-optimization, and self-protection. An autonomic system implementing the self-CHOP properties will follow a high-level policy while handling all low-level decisions without real-time oversight.

An autonomic system should always adjust to the ideal configuration, optimize its actions and planning processes to reach the most desired outcome, detect and repair faults before they cause serious harm, and proactively defend against malicious actors. This will leave human managers to simply define for the autonomic system what that desired outcome is, what faults may keep it from those outcomes, and what it must protect itself from. Automation of these key low-level tasks will also ensure that these tasks are performed promptly and properly, especially in situations in which a human will be slow to respond. The concept of autonomic computing has been applied to variety of research areas from cybersecurity [24], IoT health monitoring [25], phishing protection [26], password management [27], critical infrastructure security [28, 29] to robotics [30].

In [31], the authors describe using Large Language Models (LLMs) to realize the vision of autonomic computing (ACV). The paper discusses their work in integrating LLMs into a two-level hierarchical multi-agent framework for managing microservices. The paper focuses on the

self-optimization and self-healing capabilities of the ACV. The work also highlights how the recent advances in AI are solving some of the autonomic computing grand challenges first identified by Horn, Kephart, and Chess in their first papers that are needed to make autonomic computing a reality.

In this paper, we will lay out an approach to the real-time chess bot detection and defeat challenge utilizing an autonomic approach following the MAPE-K Architecture, as shown in Fig. 2. This architecture follows the autonomic computing reference architecture proposed by Kephart and Chess in [23].

4 Challenge

In the proposed work, we are exploring two challenges in parallel: (i) the challenge of building a chess engine which will attempt to recognize if its opponent is one of a set of open-source chess engines during the game, (ii) while also attempting to win that game. This challenge will be drawn with a few self-imposed limitations in place to limit the scope of the problem. All of the bots that we will be playing against

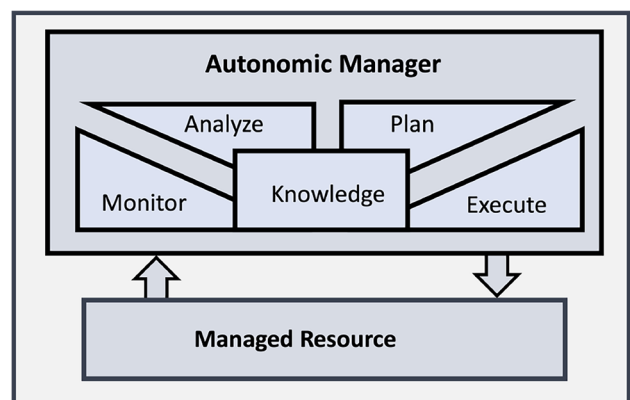


Fig. 2 MAPE-K Architecture for implementing autonomic systems, based on [23]

will be run on their default settings. While the UCI specification allows for things like variable processing times and random seeds for the engines [32], playing against bots with those features enabled would substantially increase uncertainty in the system created by each potential bot. We believe that the challenge can still be accomplished with these systems enabled; however, for the purpose of this challenge, we are more interested in playing against a large number of unique models rather than a large number of variants of the same model(s).

In this work, we do not put any weight into attempting to differentiate whether an opponent is human or automated; the focus will rather be on determining if the opponent is or is not a member of the set of known chess engines, which will be provided to the autonomic chess engine. While this will allow us to identify opponents who are utilizing the most common publicly available engines, it will not identify opponents who are utilizing a novel chess engine or who are using modified versions of these common engines. As referenced in the background section of this paper, there is fascinating work with respect to generalized engine detection, and we see potential for this project to be extended using these techniques; however, that work is not the focus of this challenge.

5 Proposed work

In the proposed work, we design and implement an autonomic chess bot that attempts the previously defined challenge. A level of self-configuration will be required for the chess bot, as we design it such that it can configure itself to play any opponent, which follows the UCI standard as an interface. The bot will exhibit self-healing by recovering from non-optimal moves made during the detection attempt

by taking advantage of new information gained from the opponent's response. Self-protection will be built into the challenge itself, as playing a series of one-off games against opponents that are immediately deleted guarantees that none of the opponent models will train on the autonomic chess bot while it is training on them. Self-optimization will be the main focus of this work, as the bot will attempt to optimize its knowledge about the opponent it is facing while still minimizing chances of losing the game.

6 Design

The data flow diagram in Fig. 3 shows a high-level overview of the design for the proposed work. As the flow diagram depicts, the design focuses on the Autonomic Chess Bot Detect and Defeat System, which uses the UCI chess interface to play against a randomly assigned opponent. The opponent is selected from the suite of potential chess bot opponents that includes the top 10 ranked open-source chess engines, as shown in the diagram in Fig. 3. Computer Chess Ratings Lists [13] website has a running list of top chess engines, and we used their rankings. Table 2 provides a snapshot of the rankings as well as Elo scores. This list is updated everyday as the engines play against each other constantly, so at the time of reading the paper, rankings and top 10 list of the engines might have changed significantly. The actual second ranking (Torch v3) and third ranking (Dragon by Komodo 3.3) chess engines are private and commercial; thus, we excluded these two highest ranking engines from our study.

Due to the popularity of chess among the software engineering community, there is an excellent environment of open-source and easy-to-use packages that enable chess

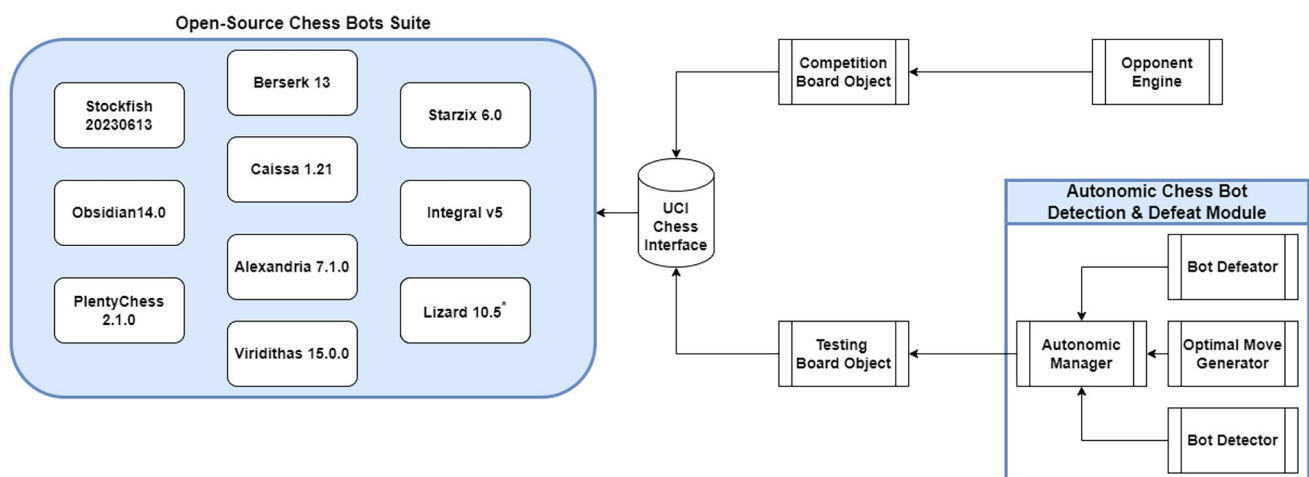


Fig. 3 A high-level data flow diagram. We replaced RubiChess engine with next high-ranking open-source engine Lizard 10.5 in our experiments

Table 2 Rankings and Elo scores of top open-source chess engines as of January 6th 2025 [13]

Rank	Name	Elo	Score
1	Stockfish 17	3643 ± 13	57.6%
2	Obsidian 14.0	3620 ± 16	53.7%
3	Berserk 13	3617 ± 10	53.0%
4	Caissa 1.21	3613 ± 16	52.9%
5	PlentyChess 2.1.0	3611 ± 13	51.3%
6	Alexandria 7.1.0	3608 ± 17	51.3%
7	Integral v6	3604 ± 20	49.5%
8	Starzix 6.0	3604 ± 16	50.8%
9	Lizard 11.1	3603 ± 18	50.9%
10	Viridithas 15.0.0	3603 ± 16	51.0%
11	[‡] RubiChess 20240112	3602 ± 12	52.0%

[‡] RubiChess had unstable runtime behavior on our windows test machines; we had to replace it with the next high-ranking open-source engine from the CCRL list

automation. For the proposed work, we utilize the package “python-chess” that provides a number of valuable tools for working with chess automation. The documentation for python-chess can be found at this address.¹ “Python-chess” critically supports the Universal Chess Interface (UCI), which is commonly used in chess engine competitions to provide a common interface between the chess engines at play and the virtual chess board [32]. Due to its use in competitions, the most highly competitive open-access chess engines are almost universally UCI compatible [13]. The proposed work takes advantage of that, running all models through UCI using default settings. This will both make the approach simpler to implement and will allow the bots tested to simulate the bots that would be quickly created by a chess cheater, who simply pulls one of the best open-source engines in the world with default settings and attempts to use it to artificially boost their level of play.

As well as serving as opponents playing on the official competition board, the autonomic system will exploit the open-source nature of these engines by querying them within a *test board object*; this will be critical information for all sub-components of the system. When it is the autonomic system’s turn to play, three sub-components will all take advantage of the available information in different ways. First, the *Optimal Move Generator* will use a highly ranked chess engine to suggest a handful of potentially optimal moves for the current board state. Next, the *Bot Detector* will be executed, attempting to narrow down the number of chess engines that the current game could be played against and suggesting potential moves that could further split the field. Finally, if

there is a high confidence that the opponent is a known bot, the *Bot Defeator* is called. The autonomic *Bot Defeator* will then utilize public information from the Computer Chess Ratings List (CCRL) [13] in order to make moves against the opponent as defined by which chess bot in the suite has the highest level of head-to-head performance against the identified opponent. The *Autonomic Manager* will take the outputs of all of the sub-components, analyze them against each other in order to understand the current state of the game, and then plan the actions it can take to optimize results from this state. The final execution will be the *Autonomic Manager* playing a move on the competition board object that will then be sent back to the competitor via UCI. All source code utilized in the implementation of this work will be made available on *GitHub*.²

The architecture shown above in Fig. 3 is intended to be highly modular, allowing many possible implementations of the *Bot Detector*, *Bot Defeator*, *Optimal Move Generator*, and *Autonomic Manager*. For this work, we have implemented simple versions of each to serve as a proof of concept for this challenge and to serve as a basis for comparison for future work. Our simple *Bot Defeator* mechanism will have a unique *Bot Defeator* assigned to each potential bot in the testing set. Every time it is the autonomic manager’s turn, each *Bot Defeator* will query its assigned bot with the board state that the opponent was last shown, an arbitrarily large number of times. From the results of this test, we can calculate the similarity percentage of these tests in which the assigned bot made the same move that the opponent made, which will be used as a similarity score. This means that for each bot, the current turn’s similarity score at to the opponent is calculated as follows:

$$TurnSimScore = \frac{num_{matchingtests}}{num_{totaltests}} \quad (2)$$

The number of total tests ($num_{totaltests}$) was selected from the set $S = \{5, 10, 20, 50, 100\}$ in the experiments to compare if more tests would yield more accurate similarity scores. The similarity score is kept as a running average, where after each move, the running similarity score is calculated as follows:

$$\rho = \frac{(\varphi \cdot numMoves) + \vartheta}{numMoves + 1} \quad (3)$$

ρ is the running similarity score, including the current turn. φ is previous running similarity score up to that point in the game. ϑ is the similarity score for the current turn, as calculated in Eq. 2. The *Autonomic Manager* will then compare

¹ <https://python-chess.readthedocs.io/en/latest/engine.html>

² <https://github.com/AndrewBengtson/AutonomicChess>

across all of the detectors, and if one bot detector carries a running similarity score that is sufficiently above the rest of the pack that bot is determined to be detected and the defeator will then be executed for that particular bot. We called this *confidence score* (γ) at the current turn of the game, which is implemented as follows in the experiments:

$$\gamma = \Delta(x, y) \cdot \text{len}(\text{botsList}) \quad (4)$$

which is basically the delta between the top 2 similarity scores times number of bots in the list (i.e., 10 in this paper).

For this, project the *Bot Defeator* is using a very simple strategy, optimal matching. Essentially, the simple Defeator used for this project simply configures the optimal strategy in order to utilize the moves determined by the opponent, which in head-to-head testing was most successful. Based on the best opponent rankings from the CCRL cite [13], the optimal opponent for each bot was Stockfish. The pseudo-code for *Autonomic Manager*, *Bot Defeator*, and *Bot Detector* are listed in Algorithms 1, 2, and 3, respectively.

This system was setup to play 50 test games, with 10 bots (see Table 2) with a variable number of tests (i.e., $S = \{5, 10, 20, 50, 100\}$). A Python script executed for 9 days (218h) on a Windows 11 Home Edition, running on a x64 based 12th Gen Intel Core i7-1260P 2.10GHz CPU, 16 GB RAM laptop, generating the following findings as explained in Section 7.

Algorithm 1 Autonomic chess manager.

Require: optimalEnginePath, testEngineDirPath, timeOutInMS, optimalMatching-Dict

Ensure: Next move for the gameboard

- 1: **Initialization:**
- 2: detectorsList, defeatorsList $\leftarrow \emptyset$.
- 3: **for** bot_i **in** testEngineDirPath **do**
- 4: botDetector.append(bot_i)
- 5: botDefeator.append(bot_i)
- 6: Initialize plotting list for each bot.
- 7: **end for**
- 8: Initialize optimalMoveGenerator with optimalEnginePath.
- 9: Set DetectionConfidenceScore to 0.
- 10: **Function:**
- 11: makeNextMove(gameBoard):
- 12: **if** DetectionConfidenceScore < 1 or gameBoard has fewer than 15 moves **then**
- 13: Track 1st and 2nd highest detection scores:
- 14: **for** each detector in detectorsList **do**
- 15: Update detection scores.
- 16: Identify highest and second-highest scores.
- 17: Append scores to plotting data.
- 18: **end for**
- 19: Compute average detector score.
- 20: Update DetectionConfidenceScore based on score difference.
- 21: Print current detection status.
- 22: **else**
- 23: **if** engineDetected is not set **then**
- 24: Set engineDetected to detected bot.
- 25: **end if**
- 26: **return** move from the detected bot's defeator.
- 27: **end if**
- 28: **return** optimal move from optimalMoveGenerator.

Algorithm 2 Bot defeator algorithm.

Require: botName, testEngineDirPath, manager

Ensure: Optimal move to defeat the bot

- 1: **Class:** botDefeator
- 2: **Attributes:**
- 3: botName: Name of the bot to defeat
- 4: manager: The chess manager object
- 5: optimalOpponentEnginePath: Path to engine
- 6: optimalEngine: Initialized to None
- 7: **Function:**
- 8: BotDefeator(botName, testEngineDirPath, manager):
- 9: Assign botName to the object attribute
- 10: Assign manager to the object attribute
- 11: Compute optimalOpponentEnginePath using testEngineDirPath and manager.optimalMatchingDict[botName]
- 12: Set optimalEngine to None
- 13: **Function:**
- 14: getOptimalMove(board):
- 15: **if** optimalEngine is None **then**
- 16: Spin up the engine using chess.engine.SimpleEngine.
- 17: popen_uci(optimalOpponentEnginePath)
- 18: Assign the engine to optimalEngine
- 19: **end if**
- 20: Use optimalEngine to compute the best move using the board state and timeOutInMS
- 21: **return** the computed move

Algorithm 3 Bot detector algorithm.

Require: botName, botOfInterestPath, manager, numAttemptsPerEval

Ensure: Detect if an opponent could be a specific bot

- 1: **Class:** BotDetector
- 2: **Attributes:**
- 3: botOfInterestPath: Path to the bot evaluated.
- 4: couldOpponentBeThisBot: True
- 5: matchingScore: None
- 6: numberOfMovesEvaluated: 0
- 7: botName: Name of the bot to detect
- 8: manager: Chess manager instance
- 9: numAttemptsPerEval: {5, 10, 20, 50, 100}
- 10: botOfInterest: Initialized engine for the bot
- 11: **Function:**
- 12: BotDetector(botName, botOfInterestPath, manager, numAttemptsPerEval)
- 13: Initialize all attributes
- 14: Start the engine with botOfInterestPath
- 15: **Function:**
- 16: couldOpponentBeThisBot(testBoard)
- 17: **if** couldOpponentBeThisBot is True **then**
- 18: **Step 1:** Extract the last move:
- 19: Pop the last move from testBoard
- 20: **Step 2:** Compare moves:
- 21: **for** attempt i in numAttemptsPerEval **do**
- 22: Generate move using botOfInterest
- 23: **if** gen-move == popped-move **then**
- 24: Increment numMatching
- 25: **end if**
- 26: **end for**
- 27: **Step 3:** Update matchingScore:
- 28: **if** matchingScore is None **then**
- 29: matchingScore $\leftarrow$ #matching/#attempts
- 30: **else**
- 31: Update matchingScore with weighted avg.
- 32: **end if**
- 33: **end if**
- 34: Increment numberOfMovesEvaluated

7 Findings

During testing, the autonomic chess detect and defeat system played 50 total games against each of the 10 open-source bots (5 times against each bot). The only difference between the sets of runs was the number of attempts to guess the move

the opponent (the bot) would make at their next turn. The number of attempts tested were $S = \{5, 10, 20, 50, 100\}$. The high-level results from these games are shown in Table 3.

Of the 50 games played, the autonomic system won 2 games, lost 6, and the rest of the games resulted in a *draw*. The 42 games that resulted in draws were due to *five-fold-repetition*

Table 3 Testing the autonomic chess detect and defeat system with 50 games against various opponents

Game#	Opponent	Detected	Winner	NumAttempts	Total moves	Detection at move
1	Alexandria	<i>Alexandria</i>	Draw	5	187	17
2	Berserk	n/a	Draw	5	76	n/a
3	Caissa	n/a	Draw	5	95	n/a
4	Integral	n/a	Draw	5	245	n/a
5	Lizard	n/a	Draw	5	56	n/a
6	Obsidian	n/a	Opponent	5	150	n/a
7	PlentyChess	n/a	Draw	5	211	n/a
8	Starzix	n/a	Draw	5	139	n/a
9	Stockfish	n/a	Draw	5	101	n/a
10	Viridithas	n/a	Opponent	5	146	n/a
11	Alexandria	n/a	Opponent	10	150	n/a
12	Berserk	n/a	Draw	10	186	n/a
13	Caissa	n/a	Draw	10	144	n/a
14	Integral	n/a	Draw	10	79	n/a
15	Lizard	n/a	Draw	10	170	n/a
16	Obsidian	n/a	Draw	10	108	n/a
17	PlentyChess	n/a	Draw	10	220	n/a
18	Starzix	n/a	Autonomic	10	235	n/a
19	Stockfish	<i>Stockfish</i>	Opponent	10	282	23
20	Viridithas	n/a	Draw	10	79	n/a
21	Alexandria	n/a	Draw	20	95	n/a
22	Berserk	n/a	Autonomic	20	267	n/a
23	Caissa	n/a	Draw	20	112	n/a
24	Integral	n/a	Opponent	20	138	n/a
25	Lizard	<i>Lizard</i>	Draw	20	398	25
26	Obsidian	n/a	Draw	20	124	n/a
27	PlentyChess	n/a	Draw	20	86	n/a
28	Starzix	n/a	Draw	20	196	n/a
29	Stockfish	<i>Stockfish</i>	Draw	20	104	57
30	Viridithas	n/a	Draw	20	95	n/a
31	Alexandria	n/a	Draw	50	158	n/a
32	Berserk	n/a	Opponent	50	170	n/a
33	Caissa	n/a	Draw	50	118	n/a
34	Integral	n/a	Draw	50	196	n/a
35	Lizard	n/a	Draw	50	137	n/a
36	Obsidian	n/a	Draw	50	179	n/a
37	PlentyChess	n/a	Draw	50	106	n/a
38	Starzix	n/a	Draw	50	210	n/a
39	Stockfish	n/a	Draw	50	210	n/a
40	Viridithas	n/a	Draw	50	210	n/a

(40 listed here for brevity)

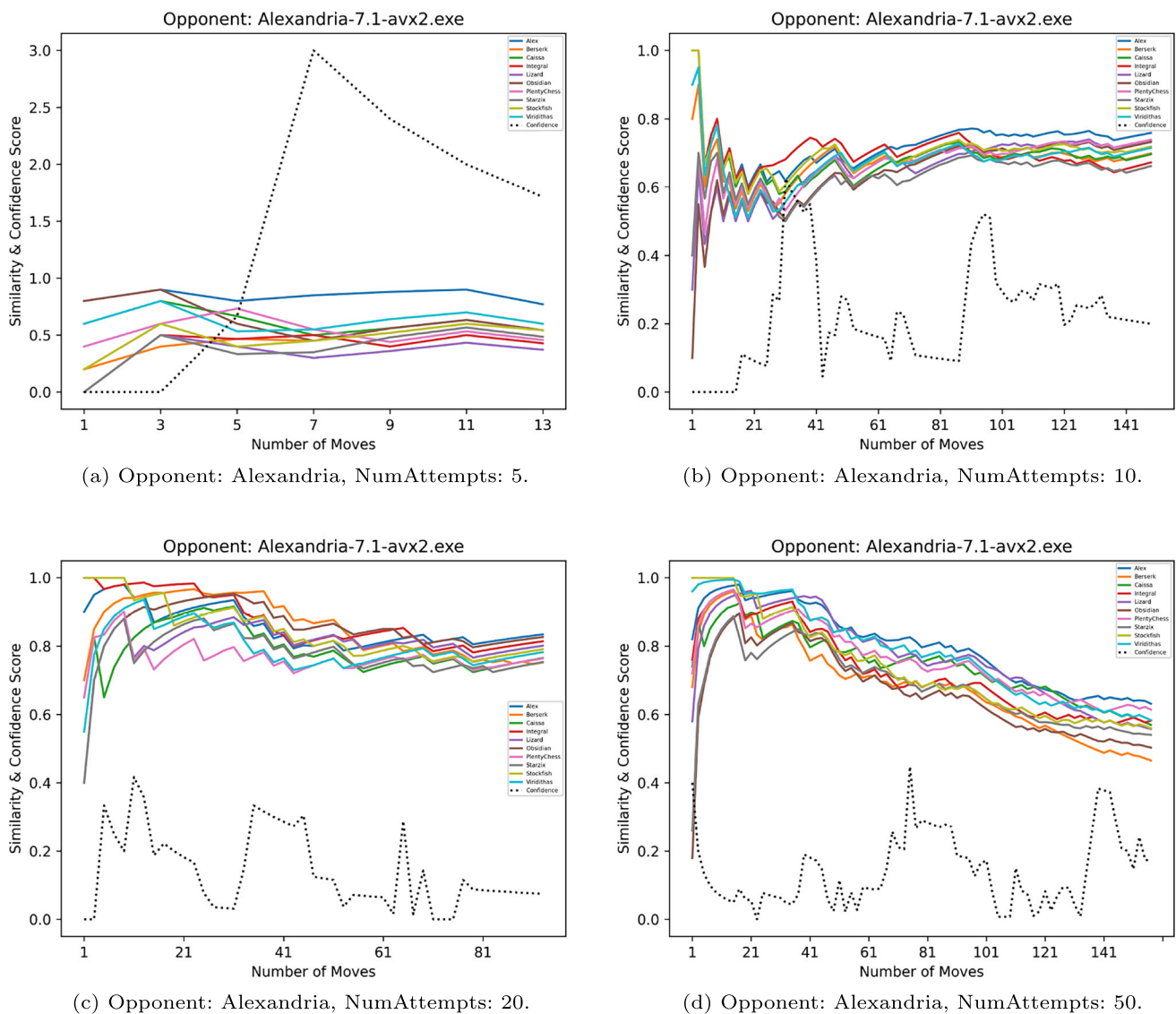


Fig. 4 Comparison of games played against opponent Alexandria. The four plots are showing 5, 10, 20, and 50 attempts to calculate the similarity score at each turn of the game. In (a), the detection was done at the 15th move (plotting stops when there is a detection), since the confidence on the similarity score was above the threshold of 1.0. After

that point, the detected engine plays against the opponent. For (b), (c), and (d), even though for most of the game Alexandria has the highest similarity score (blue line) among other bots, since the confidence score is not high enough, a detection is not announced

or *insufficient material*.³ There were 4 games in which a true positive bot detection was made; the detector was correct in all of the detections.

Diving into the details of each game a little more closely, we can review the similarity scores over time for the various bots up to the moment of detection. Figure 4 plots the change in similarity score from four selected games against the Alexandria chess engine: Game-1, Game-11, Game-21, and Game-31, which are listed in Table 3. Each plot dis-

plays the change of similarity score of the opponent playing against the autonomic manager, as well as the confidence on the detection based on the delta of the top two highest ranking bots out of the 10 bots we are evaluating. Figure 4a displays only 15 moves, since at that point, the autonomic manager decides the similarity score of the most similar engine is above the threshold, and it is well above the second highest similar engine. After that point, the autonomic manager plays against the opponent through the selected defeator engine. In sub-figures (b), (c), and (d), even though Alexandria was not above the confidence score threshold, it had the highest similarity score for most of the game (blue line). This means,

³ <https://www.chessstrategyonline.com/content/tutorials/how-to-play-chess-draws>

given a better confidence score formula, we could get better opponent bot detections during the game, given the similarity score for the correct engine overtakes the others towards the end of the game most of the time.

8 Future work

Due to the time constraints placed on the project, there is still substantial amounts of interesting work that can still be done for this challenge. For one thing, the use of reinforcement learning to create pre-trained bot detectors and defeaters seems to be an obvious improvement over the existing strategies. Furthermore, there could be benefits to increased adaptability in the weighing of the various similarity scores over time by the autonomic manager. This adaptability could further leverage the context within the game in which a decision is being made. We are also interested in expanding the scope of the detection challenge to include considerations around the specific version of the bot being detected. This will significantly expand the space of the bot detection challenge and enable more precise defeat techniques. Finally, for a fairer performance benchmark, it would be interesting to compare the performance of this system against a system which simply uses the highest-ranked chess engine with the same amount of time allotted as is taken by the autonomic system.

This approach has potential to be expanded to other domains for bot detection, such as social media, online gaming, online gambling, cyber-defense, e-Commerce, and financial services. For example, there is one estimate that 71% of Twitter accounts that retweeted stock posts were bots [33]. The technique could also be used to predict which posts were bots if examples of bot engines could be obtained or be modeled. As in the chess bot detection described above, an interaction with the social media bot could be made, and their responses could be predicted, and if there is a match, then the bot's posts could be labeled as not coming from a human and even list the probability of the detected bot. Under the topic of cyber-defense, human vs. bot detection [34] can also be addressed with our technique. A honeypot setup would record the traffic from the open internet, and our method could decide if the traffic was generated by a bot or a real human.

9 Conclusion

This work attempts to expand the field of computer chess by utilizing the self-CHOP properties and the vision of autonomic computing in order to play against and detect bots during an active chess game. The runtime approach to bot detection opens up new avenues for bot exploitation, such as

gaining knowledge of an opponent's predicted next move or exploiting known limitations of the specific opponent identified. Across competitions, humans are facing the increasing likelihood of lining up against an opponent which is potentially non-human. From traders in the markets placing bets against automated bots to social media moderators attempting to quickly detect if footage of a political candidate is authentic or the product of a deep fake, it will be increasingly important for humans to be able to detect and defeat these systems. Due to the enormous complexity of bot detection, particularly when attempting to map a bot's actions against a rising number of available alternatives, implementing an autonomic manager for these problems is highly desirable. This paper's high-level architecture for the autonomic chess detect and defeat system seems promising, with an initial simple implementation winning over half of games in an admittedly limited testing set. Substantial improvements are still possible for this system and are suggested in this paper. Given the enormous importance and complexity of this problem space, we hope that further work can expose novel solutions with greater efficiency in solving this now open challenge.

Author Contributions "A.B. wrote the code for the experiments and initial version of the manuscript. C.R. and A.T. expanded the text. All authors reviewed the manuscript."

Funding This research did not receive any funding. It was extended from a project for an Autonomic Computing class at JHU Whiting School of Engineering.

Data Availability All the source code for the experiments discussed in the paper are at this repository: <https://github.com/AndrewBengtson/AutonomicChess>.

Declarations

Conflict of interest The authors declare no competing interests.

References

1. Garcia M (2024) What Air Canada lost in 'remarkable' lying AI chatbot case. <https://www.forbes.com/sites/marisagarcia/2024/02/19/what-air-canada-lost-in-remarkable-lying-ai-chatbot-case/?sh=2810adf4696f>. Accessed March 2024
2. Bengtson A, Rouff C, Tekeoglu A (2024) An autonomic chess bot detection and defeat system. In: 2024 8th Cyber Security in Networking Conference (CSNet), Paris, France, pp 222–227. <https://doi.org/10.1109/CSNet64211.2024.10851773>
3. Laarhoven T, Ponukumati A (2023) Towards transparent cheat detection in online chess: an application of human and computer decision-making preferences. In: Computers and games: international conference, CG 2022, Virtual Event, November 22–24, 2022, Revised Selected Papers. Springer, Berlin, Heidelberg, pp 163–180. https://doi.org/10.1007/978-3-031-34017-8_14
4. Stephens E (2023) The mechanical Turk: a short history of 'artificial artificial intelligence'. *Cult Stud* 37(1):65–87. <https://doi.org/10.1080/09502386.2022.2042580>

5. Schwartz O (2019) Untold history of AI: when Charles Babbage played chess with the original mechanical Turk. <https://spectrum.ieee.org/untold-history-of-ai-charles-babbage-and-the-turk>
6. Racknitz J (1789) Ueber den Schachspieler des Herrn von Kempelen und dessen Nachbildung. <https://www.digi-hub.de/viewer/image/BV041097321/65/>
7. Elo AE, Sloan S (1978) The rating of chessplayers: past and present. ARCO Publishing Incorporated, New York, NY
8. Olesker-Taylor S, Zanetti L (2024) An analysis of Elo rating systems via Markov chains. <https://arxiv.org/abs/2406.05869>
9. Shah R (2024) Achieving the highest possible Elo rating. <https://arxiv.org/abs/2203.14567>
10. Düring B, Fischer M, Wolfram M-T (2022) An Elo-type rating model for players and teams of variable strength. *Philos Trans R Soc A Math Phys Eng Sci* 380(2224):1. <https://doi.org/10.1098/rsta.2021.0155>
11. Quan S, Yang J, Yu B, Zheng B, Liu D, Yang A, Ren X, Gao B, Miao Y, Feng Y, Wang Z, Yang J, Cui Z, Fan Y, Zhang Y, Hui B, Lin J (2025) CodeElo: benchmarking competition-level code generation of LLMs with human-comparable Elo ratings. <https://arxiv.org/abs/2501.01257>
12. Team CC. Chess ratings. <https://www.chess.com/terms/chess-ratings>. Accessed: March 2024
13. Team CCRLC (2024) CCRL 40/15 rating list. <https://computerchess.org.uk/ccrl/4040/>. Accessed: March 2024
14. Iliescu D, Monica D (2020) The impact of artificial intelligence on the chess world. *JMIR Serious Games* 8(4):24049. <https://doi.org/10.2196/24049>
15. Cohen MK. Chess: how to spot a potential cheat. <https://theconversation.com/chess-how-to-spot-a-potential-cheat-191358>. Accessed 28 Sept 2022
16. Regan KW, Macieja B, Haworth GM (2012) Understanding distributions of chess performances. In: Herik HJ, Plaat A (eds) *Advances in computer games*. Springer, Berlin, Heidelberg, pp 230–243. https://doi.org/10.1007/978-3-642-31866-5_20
17. Di Fatta G, Haworth GM, Regan KW (2009) Skill rating by Bayesian inference. In: 2009 IEEE Symposium on computational intelligence and data mining. IEEE, pp 89–94. <https://doi.org/10.1109/CIDM.2009.4938634>
18. Haworth G, Regan K, Di Fatta G (2010) Performance and prediction: Bayesian modelling of fallible choice in chess. In: Herik HJ, Spronck P (eds) *Advances in computer games: 12th International Conference, ACG 2009, Pamplona Spain, May 11–13, 2009. Revised Papers 12*. Springer, test, pp 99–110. https://doi.org/10.1007/978-3-642-12993-3_10
19. Patria R, Favian S, Caturdewa A, Suhartono D (2021) Cheat detection on online chess games using convolutional and dense neural network. In: 2021 4th International Seminar on Research of Information Technology and Intelligent Systems (ISRITI), pp 389–395. <https://doi.org/10.1109/ISRITI54043.2021.9702792>
20. Barnes DJ, Hernandez-Castro J (2015) On the limits of engine analysis for cheating detection in chess. *Comput Secur* 48:58–73. <https://doi.org/10.1016/j.cose.2014.10.002>
21. Feng X, Luo Y, Wang Z, Tang H, Yang M, Shao K, Mguni D, Du Y, Wang J (2023) ChessGPT: bridging policy learning and language modeling. <https://doi.org/10.48550/arXiv.2306.09200>
22. Horn P (2001) Autonomic computing: IBM's perspective on the state of information technology. Technical report, IBM T. J. Watson Laboratory
23. Kephart JO, Chess DM (2003) The vision of autonomic computing. *Computer* 36(1):41–50. <https://doi.org/10.1109/MC.2003.1160055>
24. Rouff C, Watkins L, Sterritt R, Hariri S (2021) SoK: autonomic cybersecurity-securing future disruptive technologies. In: 2021 IEEE International Conference on Cyber Security and Resilience (CSR), Rhodes, Greece. IEEE, pp 66–72. <https://doi.org/10.1109/CSR51186.2021.9527908>
25. Shultz M, Rouff C, Tekeoglu A (2024) Autonomic IoT pet health monitoring system. In: 2024 International Symposium on Networks, Computers and Communications (ISNCC), pp 1–6. <https://doi.org/10.1109/ISNCC62547.2024.10758996>
26. Mazur J, Rouff C, Tekeoglu A (2024) ACPPS: autonomic computing based phishing protection system. In: 2024 IEEE 14th Annual Computing and Communication Workshop and Conference (CCWC), pp 0564–0569. <https://doi.org/10.1109/CCWC60891.2024.10427857>
27. Mathurin JR, Rouff C, Tekeoglu A (2024) An autonomic computing based secure password management system. In: 2024 7th International Conference on Information and Communications Technology (ICOIAC), pp 12–17. <https://doi.org/10.1109/ICOIAC64819.2024.10913409>
28. Rubin B, Tekeoglu A, Rouff C (2024) Autonomic passive IT-OT device classification in ICS/SCADA networks. In: 2024 IEEE International Conference on Cyber Security and Resilience (CSR), pp 813–818. <https://doi.org/10.1109/CSR61664.2024.10679418>
29. Rouff C, Tekeoglu A (2024) SoK: autonomic computing based methods for ICS/SCADA and IIoT security. In: 2024 IEEE International Conference on Cyber Security and Resilience (CSR), pp 807–812. <https://doi.org/10.1109/CSR61664.2024.10679424>
30. Lewis KD, Rouff C, Tekeoglu A (2023) An autonomic architecture for multi-agent self-maintaining robotic systems. In: 2023 IEEE Intl Conf on Dependable, Autonomic and Secure Computing, Intl Conf on Pervasive Intelligence and Computing, Intl Conf on Cloud and Big Data Computing, Intl Conf on Cyber Science and Technology Congress (DASC/PiCom/CBDCCom/CyberSciTech), pp 0116–0122. <https://doi.org/10.1109/DASC/PiCom/CBDCCom/Cy59711.2023.10361421>
31. Zhang Z, Yang F, Qin X, Zhang J, Lin Q, Cheng G, Zhang D, Rajmohan S, Zhang Q (2024) The vision of autonomic computing: can LLMs make it a reality? <https://arxiv.org/abs/2407.14402>
32. Huber R, Meyer-Kahlen S (2006) Description of the universal chess interface (UCI). <https://gist.github.com/DOBRO/2592c6dad754ba67e6dcaec8c90165bf>. Accessed: March 2024
33. Ellaky Z, Benabbou F, Ouahabi S (2023) Systematic literature review of social media bots detection systems. *J King Saud Univ Comput Inf Sci* 35(5):101551. <https://doi.org/10.1016/j.jksuci.2023.04.004>
34. Udhani S, Withers A, Bashir M (2019) Human vs bots: detecting human attacks in a honeypot environment. In: 2019 7th International Symposium on Digital Forensics and Security (ISDFS), pp 1–6. <https://doi.org/10.1109/ISDFS.2019.8757534>

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.